

Software Architecture and Design Specification

Project: Chat Application (Socket Programming)

Version: 1.0

Authors: Pranay Shah (PES2UG24CS366); Nikhil Mahabala Shekar (PES2UG24CS318); Prarthana Herur (PES2UG24CS367); Nikhil B Menon (PES2UG24CS317)

Date: 14-09-2026

Status: Draft

Team Members & Contribution

Name	SRN	Contribution
Pranay Shah	PES2UG24CS366	Sections 1-2 (Introduction, Document Overview) and Section 3.1-3.2 (Goals & Constraints, Stakeholders & Concerns); overall document structuring, compilation, formatting, and final review
Nikhil Mahabala Shekar	PES2UG24CS318	Section 3.3-3.6 (Component Diagram, Component Descriptions, Architecture Pattern, Technology Stack & Data Stores)
Prarthana Herur	PES2UG24CS367	Section 3.7-3.9 (Risks & Mitigations, Traceability to Requirements, Security Architecture / STRIDE)
Nikhil B Menon	PES2UG24CS317	Section 4 (Design Overview, UML Sequence Diagrams, API Design, Error Handling/Logging/Monitoring, UX Design, Open Issues)

Revision History

Version	Date	Author	Change Summary	Approval
1.0	14-09-2026	Pranay Shah, Nikhil Mahabala Shekar, Prarthana Herur, Nikhil B Menon	Initial SAD drafted for Chat Application project	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			
Faculty Guide			

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Chat Application, a real-time messaging system built using WebSocket-based socket programming. It describes the system's component structure, chosen architecture pattern, technology stack, security design, and detailed design artifacts (sequence diagrams and API contracts) to guide implementation, review, and evaluation.

1.2 Scope

Covers the end-to-end architecture and design of real-time one-to-one and group (room-based) messaging, user presence, message persistence and history retrieval, file/media sharing, and moderation/notification support. It does not cover voice/video calling or integration with third-party enterprise messaging platforms, which remain out of scope for this version.

1.3 Audience

Developers, QA engineers, security auditors, instructors, and maintenance teams.

1.4 Definitions

- WS — WebSocket; API — Application Programming Interface; UI — User Interface
- TLS — Transport Layer Security; JWT — JSON Web Token
- DB — Database; UUID — Universally Unique Identifier
- RTT — Round-Trip Time; XSS — Cross-Site Scripting
- REST — Representational State Transfer; Pub/Sub — Publish/Subscribe messaging pattern
- RBAC — Role-Based Access Control; STRIDE — a threat-modeling framework (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege)

2. Document Overview

2.1 How to use this document

This document provides the architectural deliverables for the Chat Application project, including a UML component diagram, component descriptions, the chosen architecture pattern and rationale, technology stack, risk register, requirements traceability, a STRIDE-based threat model, UML sequence diagrams for key flows, API design, and cross-cutting design concerns (error handling, logging, monitoring, and UX).

2.2 Related Documents

- Software Requirements Specification (SRS) — Chat Application (Socket Programming)
- Software Test Plan (STP)
- Requirements Traceability Matrix (RTM), included within the SRS

3. Architecture

3.1 Goals & Constraints

Goals: Low-latency real-time delivery (target < 1 second end-to-end for online users), horizontal scalability to support growing concurrent connections, 99.5% availability, and secure-by-design handling of credentials and messages.

Constraints: Must operate within standard browser WebSocket API compatibility; limited infrastructure budget typical of a student/academic project; no dedicated PCI-DSS scope, but OWASP secure coding practices must be followed; file uploads capped to a configured size limit; must support at least 500 concurrent WebSocket connections per server instance.

3.2 Stakeholders & Concerns

- End Users: fast, reliable message delivery and privacy of their conversations
- Room Owners / Moderators: effective moderation and room-management controls

- Platform Administrators: observability, uptime, and ease of operation
- Developers: modularity, testability, and maintainability of the codebase

3.3 Component (UML) Diagram

The diagram below shows the major components of the Chat Application and their primary interactions.

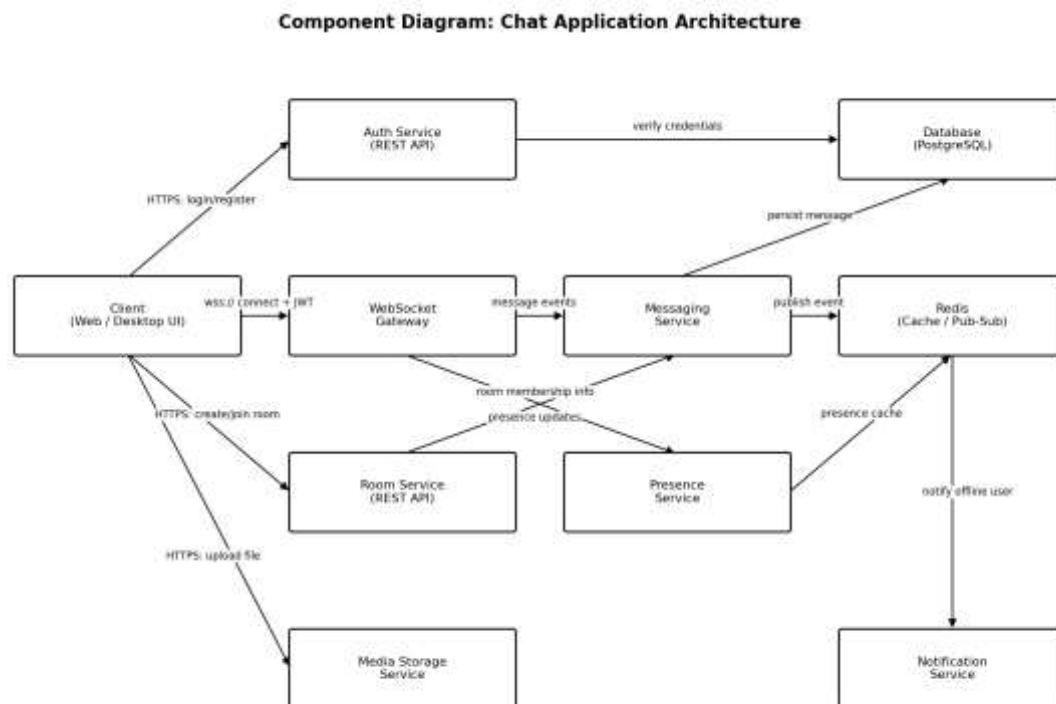


Figure 1: Component Diagram — Chat Application Architecture

3.4 Component Descriptions

Component	Description
Client (Web / Desktop UI)	Renders the chat interface; establishes and maintains the WebSocket connection; sends REST calls for auth, room, and file-upload actions.
Auth Service (REST API)	Handles user registration and login; verifies credentials against the Database; issues and refreshes JWT session tokens.
WebSocket Gateway	Manages persistent client connections; validates JWTs on connect; routes real-time events to/from the Messaging and Presence services.
Room Service (REST API)	Handles room creation, join/leave, and membership management; supplies room-membership context to the Messaging Service.
Messaging Service	Validates and processes message events; persists messages to the Database; publishes events to Redis for cross-instance broadcast.
Presence Service	Tracks online/offline/typing status per user and updates presence state, backed by the Redis cache.
Media Storage Service	Receives and stores uploaded files/images; enforces size and type restrictions on uploads.
Notification Service	Sends push notifications to offline users, triggered via Redis Pub/Sub when new messages arrive.
Database (PostgreSQL)	Durable storage for user accounts, rooms/membership, and message history.
Redis (Cache / Pub-Sub)	Caches session/presence state and provides the publish-subscribe backbone for cross-instance message and notification delivery.

3.5 Chosen Architecture Pattern and Rationale

An event-driven, service-oriented architecture (a small set of focused services behind a WebSocket gateway, sharing a Redis Pub/Sub backbone) was chosen. Real-time chat requires independently scaling connection-heavy

components (the WebSocket Gateway) separately from compute-heavy components (Messaging, Presence), which a single layered monolith would not support well under concurrent-connection load.

Rejected — Layered Monolith: simpler to build initially, but tightly couples the connection-handling layer to business logic, making it difficult to scale the WebSocket Gateway independently as concurrent users grow.

Rejected — Full Microservices with Service Mesh: offers maximum independent scalability and deployment isolation, but introduces operational overhead (service discovery, distributed tracing, mesh configuration) that is disproportionate to this project's scale and team size.

Chosen — Event-Driven Service Split: balances simplicity and scalability by grouping related responsibilities into a small number of services connected through a shared Pub/Sub backbone, enabling horizontal scaling of the WebSocket Gateway behind a load balancer while keeping the overall system easy to build, test, and operate.

3.6 Technology Stack & Data Stores

- Backend (real-time layer): Node.js with Socket.IO/ws (alternatively, Python Flask-SocketIO) for the WebSocket Gateway and Messaging Service.
- Backend (REST layer): Express.js (or Flask) for the Auth Service and Room Service APIs.
- Primary data store: PostgreSQL for users, rooms/membership, and message history.
- Cache / Pub-Sub: Redis, used for session/presence caching and cross-instance real-time event broadcast.
- Media storage: object storage (S3-compatible) for uploaded files and images.
- Transport security: TLS 1.2+ for all WebSocket (wss://) and REST (https://) traffic.
- Notifications: Firebase Cloud Messaging (optional integration) for push notifications to offline users.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
WebSocket Gateway overload under high concurrent connections	Degraded latency / dropped connections	Horizontal scaling behind a load balancer with sticky sessions; Redis Pub/Sub for cross-instance delivery
Message loss if a service instance crashes mid-delivery	Messages missing from recipient's history	Persist message to the Database before acknowledging to the sender; at-least-once delivery with de-duplication
Cross-site scripting (XSS) via malicious message content	Client-side script execution, session compromise	Server-side sanitisation/escaping of all message content before storage and rendering
Unauthorized access to rooms or moderation actions	Data leakage, abuse of moderation privileges	JWT validation on every WebSocket event and REST call; RBAC for moderator/admin actions
Network partition or client disconnection	Missed messages, poor user experience	Client-side auto-reconnect with exponential backoff; server-side offline message queuing

3.8 Traceability to Requirements

Maps key functional requirements from the SRS (IDs CHAT-F-###) to the architectural components responsible for satisfying them.

Req ID	Requirement	Component(s)
CHAT-F-002	Authenticate user	Auth Service
CHAT-F-011	Real-time direct message delivery	WebSocket Gateway, Messaging Service
CHAT-F-022	Broadcast message to room	Messaging Service, Room Service
CHAT-F-030	Online/offline presence	Presence Service
CHAT-F-041	Retrieve paginated message history	Messaging Service, Database
CHAT-F-050	File / media sharing	Media Storage Service
CHAT-F-061	Push notification to offline user	Notification Service, Redis

3.9 Security Architecture

Threat Modeling (STRIDE):

Threat Category	Example Threat	Mitigation
Spoofing	An attacker impersonates a legitimate user or service	JWT-based authentication validated on every WebSocket handshake and REST call
Tampering	Message content or tokens are altered in transit	TLS 1.2+ for all traffic; server-side validation of all message payloads
Repudiation	A user denies performing a moderation action	Audit log of moderation actions with timestamps and actor IDs
Information Disclosure	Credentials or messages are exposed to unauthorized parties	TLS in transit; salted/hashed passwords (bcrypt/argon2); least-privilege database access
Denial of Service	Excessive connections or messages degrade service	Connection and per-user message rate limiting; heartbeat-based stale-connection cleanup
Elevation of Privilege	A regular user gains moderator/admin capabilities	Server-side RBAC checks for all moderator/admin actions (never trusted from the client)

4. Design

4.1 Design Overview

The Chat Application is designed as an event-driven set of services: a WebSocket Gateway layer manages connection lifecycle and real-time event routing; a Messaging / Presence / Room service layer implements the core business logic; and a persistence layer (PostgreSQL for durable data, Redis for ephemeral/cache and cross-instance Pub/Sub) stores system state. This separation allows connection-heavy and compute-heavy components to be scaled independently and keeps each service simple to test and maintain.

4.2 UML Sequence Diagrams

Two sequence diagrams are provided below, covering the two core flows of the Chat Application: user login with WebSocket connection setup, and sending/broadcasting a group message.

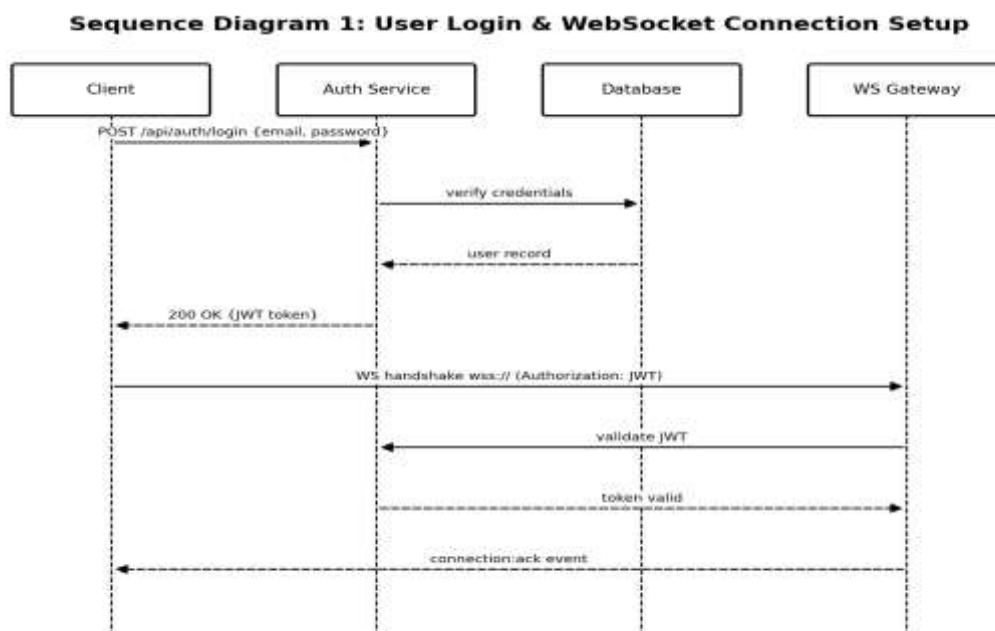


Figure 2: Sequence Diagram — User Login & WebSocket Connection Setup

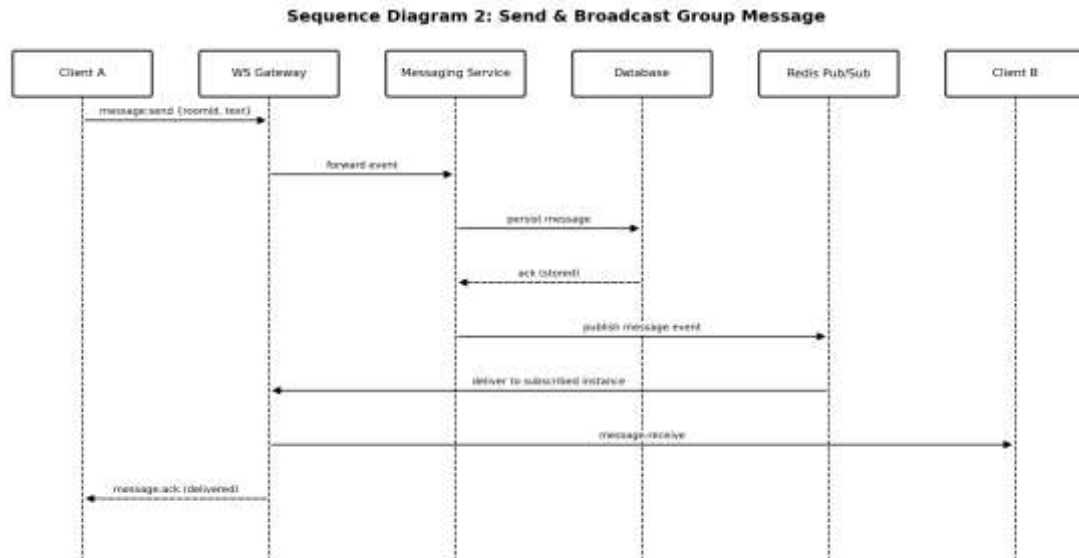


Figure 3: Sequence Diagram — Send & Broadcast Group Message

4.3 API Design

Interface definitions are provided below for three components: the Auth Service (REST), the Room Service (REST), and the real-time Messaging event interface exposed via the WebSocket Gateway.

4.3.1 Auth Service — Login

Field	Value
Endpoint	POST /api/auth/login
Component	Auth Service (REST API)
Request	{ "email": string, "password": string }
Response (200)	{ "status": "ok", "token": JWT, "expiresIn": number }
Errors	400 Malformed request; 401 Invalid credentials; 429 Too many attempts (account temporarily locked)

4.3.2 Room Service — Join Room

Field	Value
Endpoint	POST /api/rooms/{roomId}/join
Component	Room Service (REST API)
Request	Headers: Authorization: Bearer <JWT>. Body: { "inviteCode": string (optional, for private rooms) }
Response (200)	{ "status": "ok", "roomId": string, "members": [...] }
Errors	401 Invalid/expired token; 403 Invite required or invalid; 404 Room not found

4.3.3 WebSocket Gateway — Send / Receive Message

Field	Value
Event	message:send (client → server) / message:receive (server → client)
Component	WebSocket Gateway → Messaging Service
Payload (send)	{ "roomId": string, "text": string, "clientMsgId": UUID }
Payload (receive)	{ "msgId": UUID, "roomId": string, "senderId": string, "text": string, "timestamp": ISO8601 }
Errors	message:error { code: "UNAUTHORIZED" "ROOM_NOT_FOUND" "RATE_LIMITED", message }

4.4 Error Handling, Logging & Monitoring

All REST and WebSocket errors use a standardized JSON envelope of the form { "code": string, "message": string }. Logs are structured (JSON) and never contain passwords, tokens, or full message content — only metadata (user ID, event type, timestamp, status).

Monitoring metrics: active WebSocket connection count, message delivery latency (p95), message failure/error rate, database query latency, and server uptime, exposed via a metrics endpoint suitable for dashboarding and alerting.

4.5 UX Design

The chat UI follows a responsive layout with a contact/room list panel, an active conversation pane, and a message composer showing typing and delivery indicators, unread-message badges, and online/offline status. The design follows accessible practice: sufficient colour contrast, full keyboard navigation, and screen-reader-friendly labels on message controls, with support for language selection.

4.6 Open Issues & Next Steps

- End-to-end encryption for direct messages
- Voice and video calling support
- Message reactions and threaded replies
- Auto-scaling policy tuning for the WebSocket Gateway under variable load

5. Appendices

5.1 Glossary

WS, API, JWT, TLS, RBAC, Pub/Sub, RTT, REST, XSS, STRIDE.

5.2 References

- RFC 6455 — The WebSocket Protocol
- OWASP Top 10
- IEEE 42010 — Systems and Software Engineering: Architecture Description

5.3 Tools

- PlantUML, draw.io — diagramming
- Postman / Swagger — API design and testing
- Socket.IO / ws documentation — WebSocket implementation reference